

HEC-RAS Q50

Maksimum Derinlik

Otomasyonu

Bir HEC-RAS projesinde Q50 senaryosunu otomatik belirleyen ve gerçek proje sonuçlarından maksimum su derinliği haritası üreten Python uygulamasının üretim ve doğrulama sürecinin tam kaydı.

HAZIRLAYAN
Emirhan Fırtına

KURUM
Atinen Mühendislik

TARİH
29 Ağustos 2026

UYGULAMA
q50depth 1.0.0

TEKNOLOJİ
Python 3.10+ · HEC-RAS 6.6

ÇIKTI
OUTPUT/q50_depth.tif

İçindekiler

1 Bu belge nedir, nasıl okunur	3
1.1 Kanıt kuralı	3
1.2 Denetim ortamı	3
1.3 Girdi verisi	4
2 Veri setinin keşfi	6
2.1 Sonuç dosyasında hazır derinlik var mı?	6
2.2 Elde ne var: maksimum su kotu	6
2.3 Ağın gerçek hücreleri	7
3 Q50 senaryosunun belirlenmesi	8
3.1 Plan listesi nereden okunuyor	8
3.2 Alt dize araması neden yanlış	9
3.3 Sıfır ve çoklu eşleşme	9
4 HEC-RAS ile etkileşim	10
4.1 Kütüphane imzaları hatırdan değil, paketten okundu	10
4.2 Kaynak bütünlüğü: iddia değil, kanıt	11
4.3 Teslim edilen projenin durumu	12
5 Derinlik ızgarasının üretimi	14
5.1 Temel formül ve ızgaranın çözünürlüğü	14
5.2 Hücre çokgenleri	14
5.3 Arazi ve üzerindeki kot değişiklikleri	14
5.4 Islanmamış hücreler	15
5.5 İki düzeltmenin ölçülmüş etkisi	15
5.6 Koordinat sistemi	16
5.7 Denenip ölçülerek kaldırılan bir yöntem	16
6 Çıktının doğrulanması	17
6.1 Kimlik zinciri	17
6.2 GeoTIFF künyesi ve bağımsız araç	18
6.3 Girdi hidrografiyle örtüşme	19
6.4 Senaryoya özel davranmama	19
6.5 Referans çıktı ile karşılaştırma	20
6.6 Çıktıya bakıldı	21
7 Test süreci	23
7.1 Ne kadar test var, neyi kapsıyor	23
7.2 Testlerin tam listesi	23
7.3 Hata yolları ve çıkış kodları	25
7.4 Tekrarlanabilirlik	27
7.5 Testlerin teslim edilen çıktıya dokunmaması	27
8 HEC-RAS motorunun çalışmaması	28
9 Bilinen sınırlamalar ve kanıt boşlukları	29
10 Yapay zekâ kullanımı	30
11 Kanıt dizini	31

BÖLÜM 1

Bu belge nedir, nasıl okunur

Bu rapor, case study belgesinin 6. maddesinde istenen teknik açıklamanın genişletilmiş biçimidir: uygulamanın **hangi adımlardan geçerek** çıktığı ürettiğini ve her adımın **nasıl doğrulandığını** anlatır. Belgenin 7. maddesinde istenen yapay zekâ beyanı Bölüm 10 bölümündedir.

1.1 Kanıt kuralı

Bu raporda geçen her sayı ve her teknik iddia, bir komutun gerçekten çalıştırılmasıyla üretilmiştir. Komutların ham çıktısı docs/rapor/kanit/ klasöründe metin dosyası olarak durur ve bu PDF derlenirken **o dosyalardan okunur**; metin elle kopyalanmamıştır. Her kanıt bloğunun sağ alt köşesinde hangi dosyadan geldiği yazar. Sayfaya sığmayan uzun satırlar ↵ işaretiyle katlanır; hiçbir karakter atılmaz. Tek düzenleme, kişisel dizin adlarının <masaustu> ve <gecici> ile kısaltılmasıdır.

Üç kanıt sınıfı ayrı ayrı işaretlenmiştir.

(A) Yeniden üretilebilir. Bu makinede, bu belge hazırlanırken komut çalıştırılarak ölçüldü. Kanıt bloğu ham çıktıdır.

(B) Kod içinde sabitlenmiş. Otomatik testle korunuyor; test adı verilir.

(C) Başka bir makinede ölçüldü, burada tekrarlanamaz. HEC-RAS Windows programı olduğu için motorun çalıştırılmasına ilişkin bulgular bu sınıfa girer. Bu bulgular Bölüm 8 bölümünde toplanmış ve kaynakları açıkça belirtilmiştir — hiçbir (A) sınıfıymış gibi sunulmamıştır.

1.2 Denetim ortamı

Aşağıdaki bütün ölçümler tek bir makinede, tek bir sanal ortamda yapıldı.

```
$ sw_vers -productName -productVersion; uname -m
macOS
15.7.7
arm64

$ .venv/bin/python -VV
Python 3.12.13 (main, Mar 25 2026, 03:16:06) [Clang 22.1.1 ]

$ .venv/bin/python -c 'import h5py,numpy,rasterio; ...'
h5py      3.16.0 | HDF5 2.0.0
numpy     2.5.2
rasterio  1.5.1 | GDAL 3.12.4

$ git -C . log -1 --format='%h %ad %s' --date=iso
e370847 2026-08-29 23:21:45 +0300 Rapor isterlere gore sadelestirildi, PDF eklendi,
run.lo ↵
      g duzeltildi
```

Kanıt 1. Denetimin yapıldığı makine, yorumlayıcı ve kütüphane sürümleri.

kanit/A1-ortam.txt

Uygulamanın kendisi üç kütüphaneye dayanır: h5py (HEC-RAS sonuç dosyası), numpy (ızgara), rasterio (GeoTIFF). HEC-RAS'ı çalıştırmak için gereken ras-commander yalnızca Windows teslim ortamında zorunludur ve tek bir modülde kullanılır.

1.3 Girdi verisi

```
$ find CASE_DATA -type f | wc -l ; du -sh CASE_DATA
176
280M

$ find CASE_DATA -type f | sed 's/.*\\.//' | sort | uniq -c | sort -rn (uzantiya
gore)
 33 hdf
 24 g03
  9 prj
  7 tif
  6 shx
  6 shp
  6 g01
  6 dbf
  5 vrt
  5 sbx
  5 sbn
  5 h5
  4 g02
  4 dss
  2 xml
  2 x03
  2 rasmap
  2 p01
  2 o03
  2 o01

$ ls CASE_DATA/AKA_AFY_BAY_INPINAR_1/
1_Modeller

$ ls CASE_DATA/AKA_AFY_BAY_INPINAR_1/1_Modeller/ | head -40
2_CBS
A_A_B_INPINAR_1.b03
A_A_B_INPINAR_1.bco03
A_A_B_INPINAR_1.dsc.h5
A_A_B_INPINAR_1.dss
A_A_B_INPINAR_1.g01.hdf
A_A_B_INPINAR_1.g02.hdf
A_A_B_INPINAR_1.g03.hdf
A_A_B_INPINAR_1.ic.o03
A_A_B_INPINAR_1.p03.hdf
A_A_B_INPINAR_1.rasmap
A_A_B_INPINAR_1.rasmap.backup
A_A_B_INPINAR_1.u01.hdf
A_A_B_INPINAR_1.x03
A_A_B_INPINAR.b03
A_A_B_INPINAR.b04
A_A_B_INPINAR.b05
A_A_B_INPINAR.b06
A_A_B_INPINAR.b07
A_A_B_INPINAR.bco01
A_A_B_INPINAR.bco03
A_A_B_INPINAR.bco04
A_A_B_INPINAR.bco05
A_A_B_INPINAR.bco06
A_A_B_INPINAR.bco07
A_A_B_INPINAR.dsc.h5
A_A_B_INPINAR.dss
A_A_B_INPINAR.f01
A_A_B_INPINAR.g01
A_A_B_INPINAR.g01.hdf
A_A_B_INPINAR.g02
A_A_B_INPINAR.g02.hdf
```

```
A_A_B_INPINAR.g03  
A_A_B_INPINAR.g03.hdf  
A_A_B_INPINAR.ic.001  
A_A_B_INPINAR.ic.003  
A_A_B_INPINAR.ic.004  
A_A_B_INPINAR.ic.005  
A_A_B_INPINAR.ic.006  
A_A_B_INPINAR.ic.007
```

Kanıt 2. Teslim edilen veri setinin envanteri. Belgede bildirilen konum doğrulandı.

kanit/A2-veri-envanteri.txt

176 dosya, 280 MB. Belgenin 2. maddesi veri konumunu CASE_DATA\\AKA_AFY_BAY_INPINAR_1\\1_Modeller olarak veriyor; envanterdeki klasör yapısı bununla birebir örtüşüyor.

BÖLÜM 2

Veri setinin keşfi

Belgenin 2. maddesi hazır bir maksimum derinlik çıktısı sağlanmadığını söylüyor ve adayın dosya ilişkilerini kendisinin belirlemesini bekliyor. İlk iş, sonuç dosyasının gerçekten ne içerdiğini ölçmektir.

2.1 Sonuç dosyasında hazır derinlik var mı?

Plan sonuç dosyası A_A_B_INPINAR.p05.hdf bir HDF5 arşividir. İçindeki bütün yollar tarandı ve adında “depth” geçen her veri seti listelendi.

```
$ python -c "h5py ile p05.hdf icindeki TUM yollari say"
dosya: A_A_B_INPINAR.p05.hdf | 3.8 MB
toplaml yol      : 439
grup             : 118
veri seti        : 321

adinda 'depth' gecen veri seti sayisi: 1
Event Conditions/Unsteady/Boundary Conditions/Normal Depths/2D: inpinar BCLine:
outflow
w
```

Kanıt 3. Sonuç dosyasındaki 439 yolun tamamı tarandı. Tek “depth” eşleşmesi bir sınır koşuludur kanit/A3-hdf-yollari.txt (mansap normal derinliği), derinlik rasteri değildir.

Bu, raporun en belirleyici bulgusudur: **HEC-RAS hazır bir maksimum derinlik veri seti üretmiyor.** Derinlik türetilmek zorunda.

2.2 Elde ne var: maksimum su kotu

```
$ Summary Output altındaki veri setleri:
Results/Unsteady/Output/Output Blocks/Base Output/Summary Output/2D Flow Areas/
inpinar
Cell Cumulative Iter Lookup          shape=(5, 2)      dtype=int32
Cell Cumulative Iteration            shape=(5667,)     dtype=float32
Cell Last Iteration                  shape=(5667,)     dtype=int32
Cell Maximum Water Surface Error      shape=(2, 5667)   dtype=float32
Maximum Face Velocity                shape=(2, 12868)  dtype=float32
Maximum Water Surface                 shape=(2, 5667)   dtype=float32
Minimum Face Velocity                shape=(2, 12868)  dtype=float32
Minimum Water Surface                 shape=(2, 5667)   dtype=float32
Starting Differences Velocity          shape=(3, 12868)  dtype=float32
Starting Differences WSE              shape=(3, 5667)   dtype=float32

$ Maximum Water Surface ayrıntısı
shape : (2, 5667) dtype: float32
attr  : Max Time = 0.04859953746199608
attr  : Max Value = 1376.474609375
attr  : Min Time = 0.0
attr  : Min Value = 1323.98388671875
attr  : Rows Variables = [b'WSEL' b'Time']
attr  : Units = [b'm' b'days']
satir 0 (su kotu, m) ilk 6 : [1369.383 1368.605 1368.53 1368.275 1367.703
1366.904]
satir 1 (zaman, saat) ilk 6: [0. 0. 0. 0. 0. 0.]
kot araligi : 0.0 -> 1376.475 m

$ Hucre taban kotlari (karsilastirma icin)
shape: (5667,) araligi: 1323.322 -> 1376.474 m
```

```
$ Su kotu == taban kotu olan hucre sayisi (hic islanmamis):
kuru : 2586 / 5667 |  islak: 3081
```

Kanıt 4. Özet çıktı veri setleri ve Maximum Water Surface'ın içeriği. Birim ve satır adları kanıt/A4-maksimum-su-kotu.txt HDF özniteliklerinden okundu, varsayılmadı.

Veri seti 2×5667 : birinci satır su yüzeyi **kotu** (m), ikinci satır o kotun görüldüğü zaman (gün). Rows Variables özniteliği bunu kendisi söylüyor — [WSEL, Time] — yani sıralama tahmin edilmedi.

Aynı kanıt bloğunun son satırı ileride kritik olacak bir gerçeği de gösteriyor: hiç ıslanmamış hücreler için HEC-RAS maksimum su kotunu **hücrenin kendi taban kotu** olarak raporluyor. Bu, su değildir; Bölüm 5.4 bölümünde ele alınıyor.

2.3 Ağın gerçek hücreleri

Hücre dizileri 5667 uzunluğunda, ama bunların hepsi gerçek hücre değil.

```
$ Mesh: inpinar
ham hucre dizisi uzunlugu (cell_count) : 5667
>=3 kose noktası olan                : 5470
taban kotu NaN olmayan                : 5470
ikisi birden = gercek hucre          : 5470
hayalet (ghost) hucre                : 197

$ Islanma filtresi: max_water_surface > cell_min_elevation
islak hucre : 4165 / 5470
kuru hucre  : 1305

$ Filtre olmasaydi: gercek hucrelerin hepsi boyanirsa
hucre bazli derinlik min / maks : -0.0023 / 1.4197 m
<= 0 olan (su degil)           : 1305 hucre
filtre sonrasi hucre derinligi ort / maks : 0.1534 / 1.4197 m

$ Sonuc dosyasinin kendi kimlik alanlari (dosya adindan bagimsiz)
plan_title           = 'A_A_B_INPINAR_Q50'
plan_short_id        = 'Q50'
plan_filename        = 'A_A_B_INPINAR.p05'
geometry_filename    = 'A_A_B_INPINAR.g03'
flow_filename        = 'A_A_B_INPINAR.u05'
program_version      = 'HEC-RAS 6.6 September 2024'
simulation_window     = '02May2025 01:00:00 to 02May2025 02:10:00'
terrain_filename     = '.\merge.Clone.hdf'
terrain_layer        = 'merge.Clone'

$ Projeksiyon WKT (ilk 240 karakter)
PROJCS["WGS_1984_Transverse_Mercator",GEOGCS["GCS_WGS_1984",DATUM["D_WGS_1984",SPHEROID[↵
"WGS_1984",6378137.0,298.257223563]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.0174532925199
↵
433]],PROJECTION["Transverse_Mercator"],PARAMETER["false_easti
```

Kanıt 5. Hayalet hücrelerin sayımı ve ıslanma filtresinin hücre düzeyindeki etkisi. Kimlik alanları kanıt/B1-mesh-ve-kimlik.txt da dosyadan okundu.

HEC-RAS hücre dizilerini sınırda “hayalet” hücrelerle dolduruyor: üçten az köşe noktası ve NaN taban kotu taşıyorlar. 5667 girdinin **197’si** hayalet, **5470’i** gerçek hücre. Uygulamanın Mesh.real_cells özelliği bu iki koşulu birlikte uyguluyor; ilgili test test_ghost_cells_are_excluded.

BÖLÜM 3

Q50 senaryosunun belirlenmesi

Belgenin 4. maddesi Q50'nin manuel yazılmasını veya kullanıcıya seçtirilmesini yasaklıyor. Doğru planı bulmak ilk bakışta bir metin araması gibi görünüyor; uygulamada iki ayrı tuzak barındırıyor.

3.1 Plan listesi nereden okunuyor

```
$ grep -n 'Plan File=\\Current Plan=' A_A_B_INPINAR.prj
2:Current Plan=p05
13:Plan File=p03
14:Plan File=p02
15:Plan File=p01
16:Plan File=p04
17:Plan File=p05
18:Plan File=p06
19:Plan File=p07

$ ls $D/*.p?? | xargs -n1 basename      (diskteki plan dosyalari)
A_A_B_INPINAR.p01
A_A_B_INPINAR.p02
A_A_B_INPINAR.p03
A_A_B_INPINAR.p04
A_A_B_INPINAR.p05
A_A_B_INPINAR.p06
A_A_B_INPINAR.p07
Backup.p01

$ head -3 $D/Backup.p01      (proje dosyasinda BILDIRILMEYEN dosya)
Plan Title=A_A_B_INPINAR_Q50
Program Version=6.60
Short Identifier=Q50

$ head -3 $D/A_A_B_INPINAR.p05  (dogru plan)
Plan Title=A_A_B_INPINAR_Q50
Program Version=6.60
Short Identifier=Q50
```

Kanıt 6. Plan listesinin kaynağı ve diskteki plan dosyaları. Backup.p01, proje dosyasında **bildi-** kanit/C1-plan-kaynagi.txt **rilmediği hâlde** p05 ile birebir aynı kimliği taşıyor.

Kanıtın son iki bloğu tuzağı tek başına anlatıyor: Backup.p01 ile A_A_B_INPINAR.p05 dosyalarının ilk satırları aynı — Plan Title=A_A_B_INPINAR_Q50, Short Identifier=Q50. Klasörü *.p?? deseniyle tarayan bir seçici **iki** tam eşleşme bulur ve hangisinin doğru olduğunu bilemez. Ne kadar iyi bir kalıp yazılırsa yazılsın bu ayırım yapılamaz, çünkü metin gerçekten aynıdır.

Alınan karar: plan listesi klasör taramasıyla değil, A_A_B_INPINAR.prj dosyasındaki Plan File= satırlarından okunur. Proje dosyası yalnızca p01–p07'yi bildirir; Backup.p01 orada yer almadığı için hiç değerlendirmeye girmez. Tuzak bir istisna ile değil, **doğru kaynağı seçerek** çözülür. İlgili test: test_plan_list_comes_from_the_project_file_not_the_folder.

Proje dosyasında Current Plan=p05 de yazıyor ve bu seçimle uyuyor. Bu alan **seçim ölçütü olarak kullanılmadı**: projeyi en son kimin nasıl kapattığına bağlı bir durumdur, belgenin istediği otomatik belirleme değildir. Yalnızca çalışma kaydına “seçimle uyuyor” notu olarak düşülür; üretim koşusunun tam kaydı Bölüm 6 bölümündedir.

3.2 Alt dize araması neden yanlış

```
$ scenario_pattern('Q50') -> (?![0-9])[Qq]0*50(?![0-9])
```

metin	alt dize (naif)	sinir kontrollu	
Q50	True	True	
Q050	False	True	<-- FARK
q50	True	True	
A_A_B_INPINAR_Q50	True	True	
INPINAR_Q500_UNSTEADY	True	False	<-- FARK
A_A_B_INPINAR_Q500	True	False	<-- FARK
Q100	False	False	
Q1000	False	False	
Q5000	True	False	<-- FARK
Q50A	True	True	
XQ50	True	True	
Q5	False	False	
AQ50B	True	True	

Kanıt 7. Sınır kontrollü kalıp ile naif alt dize aramasının aynı girdiler üzerinde karşılaştırılması. “FARK” işaretli kanıt/C2-regex.txt satırlar naif yaklaşımın hata verdiği yerlerdir.

Dört ayrım noktası var. Naif arama Q500 ve Q5000 içinde Q50 görüp yanlış eşleşiyor; buna karşılık Q050 yazımını kaçırıyor. Kalıptaki 0* sıfır dolgusunu kabul ederken, iki yandaki (?![0-9]) ve (?![0-9]) komşu rakamı reddediyor.

Bu davranış TestScenarioPattern sınıfındaki 13 testle sabitlenmiştir (Bölüm 7); ayrıca gerçek veri üzerinde --scenario Q050 komutu da p05'i seçer.

3.3 Sıfır ve çoklu eşleşme

Hiçbir durumda “ilkini al” davranışı yoktur. Bu projede --scenario Q500 iki planla eşleştiği için bilinçli olarak reddedilir; kanıtı Bölüm 7.3 bölümündedir.

BÖLÜM 4

HEC-RAS ile etkileşim

HEC-RAS bir Windows programıdır; bu, boru hattındaki tek platforma bağlı adımdır ve `q50depth/compute.py` modülüne hapsedilmiştir.

4.1 Kütüphane imzaları hatırdan değil, paketten okundu

Belgenin 2. maddesi `ras-commander` kütüphanesini öneriyor. Kütüphanenin fonksiyon adları ve parametreleri dokümandan veya hatırdan yazılmadı; paket kurulum `inspect.signature` ile okundu. Aşağıdaki blok bu okumanın **bu belge hazırlanırken tekrarlanmış** hâlidir.

```
$ import ras_commander; inspect.signature ile imzalar okunuyor
surum: 0.99.1

init_ras_project(ras_project_folder, ras_version=None, ras_object=None,
load_results_successfully=True, hide_intro=False, accept_tcu=False) -> 'RasPrj'
RasCmdr.compute_plan(plan_number: Union[str, numbers.Number, pathlib.Path],
dest_folder: Optional[str] = None, ras_object=None, clear_geompre=False, force_geompre: bool = False,
force_rerun: bool = False, num_cores=None, overwrite_dest=False, skip_existing: bool = False,
verify: bool = False, stream_callback: Optional[Callable] = None, use_optimal_hdf_settings:
bool = False, hdf_settings_profile: str = 'balanced', hdf_additional_variables:
Optional[List[str]] = None, hdf_output_variables: Optional[List[str]] = None, hdf_output_options:
Optional[Dict[str, Any]] = None, hdf_output_profile: Optional[str] = None, dialog_watchdog:
bool = True) -> 'ComputeResult'
RasControl.run_plan(plan: Union[str, pathlib.Path], ras_object=None,
force_recompute: bool = False, use_watchdog: bool = True, max_runtime: int = 86400,
refresh_results: bool = True) -> 'RasControlResult'

$ COM ProgID (RasControl kaynak kodundan):
HECRASController API wrapper with ras-commander style interface.
```

Kanıt 8. `ras-commander 0.99.1`'in gerçek imzaları, kurulu paketten okundu.

kanit/H1-ras-commander.txt

Bir adım daha atıldı: uygulamanın kullandığı her parametre adının imzada gerçekten bulunup bulunmadığı tek tek sınandı.

```
$ COM ProgID araniyor:
... (12 baska surum icin ProgID)
RAS66.HECRASController
RAS67.HECRASController

$ Uygulamanin kullandigi parametre adlari gercekten var mi?
init_ras_project(ras_version=...) -> VAR
RasCmdr.compute_plan(plan_number=...) -> VAR
RasCmdr.compute_plan(num_cores=...) -> VAR
RasCmdr.compute_plan(dest_folder=...) -> VAR
RasCmdr.compute_plan(overwrite_dest=...) -> VAR
RasControl.run_plan(plan=...) -> VAR
```

```

RasControl.run_plan(force_recompute=...) -> VAR

$ compute.py bu cagrilari nasil yapiyor (kaynak):
rc = _import_ras_commander()
ras_version = _resolve_ras_version(ras_dir)
short_number = plan_number.lstrip("pP") # ras-commander addresses plans as "05"

started = time.monotonic()
try:
    project = rc.init_ras_project(str(project_folder), ras_version)
except Exception as exc:
    raise ComputeError(
        f"HEC-RAS project {prj_path.name} could not be initialised: {exc}",
        hint="Check that HEC-RAS is installed at the path given by --ras-dir.",
    ) from exc

try:
    if runner == "cmdr":
        outcome = rc.RasCmdr.compute_plan(
            short_number,
            ras_object=project,
            force_rerun=True,
            num_cores=cores,
        )
        succeeded = bool(getattr(outcome, "success", outcome))
        detail = f"RasCmdr.compute_plan -> success={succeeded}"
    else:
        outcome = rc.RasControl.run_plan(
            short_number,
            ras_object=project,
            force_recompute=True,
        )
        succeeded = bool(getattr(outcome, "success", outcome))
        detail = f"RasControl.run_plan -> success={succeeded}"
except Exception as exc:
    raise ComputeError(
        f"HEC-RAS failed while computing plan {plan_number}: {exc}",
        hint="The HEC-RAS computation log in the project folder "
    )

```

Kanıt 9. Uygulamanın kullandığı parametre adları kütüphanenin imzasıyla karşılaştırıl- kanit/H2-api-uyum.txt
dı; RAS66.HECRASController ProgID'sinin kütüphanede desteklendiği doğrulandı. Altta
compute.py'nin ilgili bölümü.

Kullanılan iki yol şudur:

Yol	Çağrı	Ne yapar
Varsayılan	RasCmdr.compute_plan()	HEC-RAS komut satırı çalıştırıcısı
--runner controller	RasControl.run_plan()	RAS66.HECRASController COM otomasyonu – ara- yüzün kullandığı arabirim

4.2 Kaynak bütünlüğü: iddia değil, kanıt

Belgenin 4. maddesi orijinal proje dosyalarının değiştirilmemesini şart koşuyor. HEC-RAS bir planı çalıştırdığında proje klasörüne yazar; bu yüzden uygulama proje ağacını önce bir çalışma dizinine kopyalar ve hesabı orada yapar.

“Değiştirmedim” demek yetmez. Uygulama çalışma başında ve sonunda kaynak klasörün SHA-256 parmak izini çıkarıp karşılaştırır. Peki bu kontrol gerçekten bir şey yakalıyor mu? Aşağıdaki kontrollü deney bunu sınıyor: gerçek veri **kopyalanıyor**, kopya üzerinde önce hiçbir şey değiştirilmiyor, sonra tek bir bayt ekleniyor, sonra bir dosya siliniyor.

```

$ Gercek veri KOPYALANIYOR (orijinale dokunulmuyor)
  kopya: <gecici>
  parmak izi alındı: 176 dosya

$ 1) Hicbir sey degistirilmeden:
  source unchanged (176 files, full check)

$ 2) Tek bir dosyaya tek bir bayt eklendikten sonra:
  SOURCE CHANGED -> 1 modified: AKA_AFY_BAY_INPINAR_1/1_Modeller/A_A_B_INPINAR.prj

$ 3) Bir dosya silindikten sonra:
  SOURCE CHANGED -> 1 modified: AKA_AFY_BAY_INPINAR_1/1_Modeller/A_A_B_INPINAR.prj;
1 rem ↵
  oved: AKA_AFY_BAY_INPINAR_1/1_Modeller/A_A_B_INPINAR.p01

$ kopya silindi. Gercek CASE_DATA'ya hic yazilmadi.

```

Kanıt 10. Bütünlük kontrolünün boş bir vaat olmadığına kanıtı. Deney bir kopya üzerinde yapıldı; orijinal kanıt/D2-butunluk.txt CASE_DATA klasörüne yazılmadı.

Tek baytlık değişiklik de, silinen dosya da yakalandı. --integrity full her dosyanın SHA-256 özetini alır; fast yalnızca boyut ve zaman damgasına bakar.

4.3 Teslim edilen projenin durumu

Proje neyi bildiriyor, her plan neye ihtiyaç duyuyor, diskte ne var — bu üçünün ayrıştığı noktalar tools/audit_project.py ile tarandı.

```

Audit of <masaustu>/CASE_DATA

BLOCKER (8)
-----
plan p03                                uses unsteady flow u01, which the project file does
not dec ↵
  lare (the file exists on disk)
plan p04 inflow                          boundary condition reads ..\..\akarcay_debi.dss,
which does ↵
  not resolve
plan p05 inflow                          boundary condition reads .
\_CBS\akarcay_debiler\akarcay_deb ↵
  i.dss, which does not resolve
plan p06 inflow                          boundary condition reads .
\_CBS\akarcay_debiler\akarcay_deb ↵
  i.dss, which does not resolve
plan p07 inflow                          boundary condition reads .
\_CBS\akarcay_debiler\akarcay_deb ↵
  i.dss, which does not resolve
geometry g01                             A_A_B_INPINAR.g01.hdf lacks Geometry/Structures/
Property Ta ↵
  bles, Geometry/GeomPreprocess; the unsteady engine reads these and crashes
without them
geometry g02                             A_A_B_INPINAR.g02.hdf lacks Geometry/Structures/
Property Ta ↵
  bles, Geometry/GeomPreprocess; the unsteady engine reads these and crashes
without them
geometry g03                             A_A_B_INPINAR.g03.hdf lacks Geometry/Structures/
Property Ta ↵
  bles, Geometry/GeomPreprocess; the unsteady engine reads these and crashes
without them

WARNING (23)
-----
plan p03 output                          DSS destination dss does not resolve

```

```

plan p02 output      DSS destination dss does not resolve
plan p01 output      DSS destination dss does not resolve
plan p04 output      DSS destination dss does not resolve
plan p05 output      DSS destination .\Q50\Q50.dss does not resolve

```

Kanıt 11. Denetim aracının çıktısı: sekiz engelleyici bulgu.

kanit/veri-denetimi-ciktisi.txt

Sekiz engelleyicinin özeti: p03 bildirilmemiş bir akış dosyası kullanıyor; p04–p07 giriş debisi DSS’ini çözemiyor (p05 ._CBS\\... altında arıyor, klasörün diskteki adı 2_CBS); g01–g03 geometri dosyalarında ön işlenmiş tablolar yok. Ayrıca dört akış dosyasının hiçbirinde hidrograf gömülü değil. Tam liste docs/VERI-DENETIMI.md dosyasındadır.

Uygulama bu eksikleri yalnızca çalışma kopyasında onarır ve her onarımı çalışma kaydına yazar. Çözülmeyen bir giriş dosyası için proje ağacında aynı adlı dosya aranır; **tam bir tane** bulunursa modelin beklediği yola kopyalanır. Hiç ya da birden fazla aday çıkarsa tahmin yürütülmez, anlamlı hatayla durulur – yanlış hidrografla hesaplanmış bir taşkın haritası, hesaplanmamış olmasından kötüdür. İlgili testler test_references.py içinde yedi adettir.

BÖLÜM 5

Derinlik ızgarasının üretimi

5.1 Temel formül ve ızgaranın çözünürlüğü

$\text{derinlik(piksel)} = \text{pikselli kaplayan hücrenin maksimum su kotu} - \text{o pikseldeki arazi kotu}$

Izgara hücre çözünürlüğünde değil **arazi çözünürlüğünde** kurulur (bu veri setinde 0,1 m), çünkü bir HEC-RAS 2D hücresi düz değildir; içinde alt-ızgara topografya taşır. Pencere, arazi rasterinin kendi piksel ızgarasına hizalanır (_grid_aligned_to), böylece arazi yeniden örneklenmeden okunur.

5.2 Hücre çokgenleri

Cells FacePoint Indexes bir hücrenin köşelerini verir ama halka sırasında vermez. Köşeler hücre merkezine göre açıya sıralanır; HEC-RAS 2D hücreleri dışbükey olduğu için bu sıralama tam olarak çokgen sınırdır.

5.3 Arazi ve üzerindeki kot değişiklikleri

Geometri merge.Clone arazisi üzerine kuruludur. RASMapper arazi katmanı bir **çifttir**: .vrt kot ızgarasını, .hdf ise üzerine çizilen değişiklikleri tutar. Bu değişiklikler .vrt'ye gömülü değildir; RASMapper onları anlık uygular.

```
$ Sonuc dosyasi arazi olarak neyi soyluyor?
  terrain_filename : '.\\merge.Clone.hdf'
  terrain_layer    : 'merge.Clone'

$ terrain.resolve(...) ne buldu?
  raster_path      : merge.Clone.vrt
  modifications    : 69 adet
  tipleri          : {'Add': 69}
  degerleri (m)    : {20.0: 69}
  katmanlar        : {'buildings': 69}

$ Ilk uc degisiklik poligonu:
  layer='buildings' kind='Add' value=20.0 halka=1 kose=5 ilk kose=[578128.41,
4304515.48 ↵
  ]
  layer='buildings' kind='Add' value=20.0 halka=1 kose=5 ilk kose=[578122.82,
4304532.58 ↵
  ]
  layer='buildings' kind='Add' value=20.0 halka=1 kose=5 ilk kose=[578291.3,
4304637.21]

$ Arazi rasteri: merge.Clone.vrt
  boyut           : 54679 x 39423
  cozunurluk      : 0.1 m
  nodata          : -9999.0 | dtype: float32 | crs:
PROJCS["WGS_1984_Transverse_Mercator",GEOG ↵
  CS["WGS_84",DATUM["WGS_1984",SPHEROID["WGS
84",6378137,298.257223563,AUTHORITY["EPSG","703 ↵
0"]],AUTHORITY["EPSG","6326"]],PRIMEM["Greenwich",0],UNIT["degree",0.0174532925199433,AUTH
  ORITY["EPSG","9122"]],AUTHORITY["EPSG","4326"]],PROJECTION["Transverse_Mercator"],PARAMETE
  R["latitude_of_origin",0],PARAMETER["central_meridian",30],PARAMETER["scale_factor",1],PAR
  AMETER["false_easting",500000],PARAMETER["false_northing",0],UNIT["metre",1,AUTHORITY["EPS
```

```
G", "9001"]], AXIS["Easting", EAST], AXIS["Northing", NORTH]]
```

Kanıt 12. Arazi katmanının çözülmesi ve kot değişikliklerinin sayımı — 69 poligon, hepsi Add tipinde, kanit/B3-arazi.txt hepsi +20 m, hepsi buildings katmanında.

Yalnızca .vrt okunursa harita binaların üstüne metrelerce su boyar. Etkisi Bölüm 5.5 bölümünde ölçülüyor.

5.4 Islanmamış hücreler

Hiç ıslanmayan bir hücre için HEC-RAS maksimum su kotunu hücrenin kendi taban kotu olarak raporlar. Tabanı yükseltilmiş bir bina üzerinde oturan kuru hücre, binanın kaplamadığı şeritte metrelerce sahte derinlik üretir. Bu yüzden yalnızca

maksimum su kotu > hücre taban kotu

olan hücreler boyanır (Mesh.wet_cells). Filtre 5470 gerçek hücrenin 4165'ini ıslak sayıyor.

5.5 İki düzeltmenin ölçülmüş etkisi

İki düzeltme de “gerekli” diye değil, **ölçülerek** konuldu. Aşağıdaki deney dördünü birden kurar: bina kot değişikliği açık/kapalı × kuru hücre filtresi açık/kapalı.

\$ İki düzeltmenin tam carpım tablosu (2x2)

bina kotu	kuru hücre filtresi	ıslak piksel	maks m	ort m	>2m piksel
UYGULANMADAN	KAPALI	522960	20.201	3.391	87012
UYGULANMADAN	ACIK	469610	20.201	1.891	42151
UYGULANARAK	KAPALI	418510	19.611	0.130	36
UYGULANARAK	ACIK	413858	1.625	0.130	0

Son satır = teslim edilen yapılandırma.

Kanıt 13. İki düzeltmenin tam carpım tablosu. Son satır teslim edilen yapılandırmadır.

kanit/B5-carpim-tablosu.txt

Tablo, iki düzeltmenin **farklı belirtileri** iyileştirdiğini gösteriyor:

Düzeltilme	Neyi düzeltiyor	Ölçülen etki
Bina kot değişikliği	ortalama derinlik	3,391 m → 0,130 m
Kuru hücre filtresi	maksimum derinlik	19,611 m → 1,625 m
İkisi birlikte	2 m üstü piksel sayısı	87.012 → 0

Bina kot değişikliği uygulanmadığında ortalama derinlik 3,391 m çıkıyor; bu, 70 dakikalık ve tepe debisi 1,69 m³/s olan bir olay için fiziksel olarak imkânsız. Kuru hücre filtresi olmadan ise en derin piksel 19,611 m'ye çıkıyor — bu sayı tesadüfi değil, binalara eklenen 20 m'nin ta kendisidir.

Daha önceki bir sürümdeki düzeltme. Bu raporun HTML sürümünde 3,37 m değeri yalnızca bina kot değişikliğine atfedilmişti. Yukarıdaki ölçüm, o değer **iki düzeltme de kapalıyken** geçerli olduğunu gösteriyor; bina düzeltmesi tek başına ortalama 0,130 m'ye indiriyor. Aynı şekilde “36–54 piksel, en fazlası 19,96 m” ifadesi yerine ölçülen değerler 36 piksel ve 19,611 m'dir. Rakamlar bu raporda ölçüme göre düzeltilmiştir.

Bu iki kuralı koruyan testler:

- test_building_modification_is_applied_to_the_terrain
- test_without_the_dry_cell_rule_the_building_would_flood
- test_dry_cell_on_a_raised_footprint_is_not_wet
- test_building_inside_a_wet_cell_stays_dry

5.6 Koordinat sistemi

CRS, sonuç dosyasının kök Projection özneliğindeki WKT'den okunur. Elle EPSG:32636 yazmak cazipti ve yanlış olurdu.

```
$ 0,117 m sayısı nereden geliyor?
  ıslak hücrelerin ortalaması : 0.1534 m
  TUM gerçek hücrelerin ortalaması (kuru=0 sayılır): 0.1168 m   <- rapordaki 0,117
  kontrol: 0.1534 * 4165 / 5470 = 0.1168

$ Projeksiyon WKT'sinin tamamı:
PROJCS["WGS_1984_Transverse_Mercator",GEOGCS["GCS_WGS_1984",DATUM["D_WGS_1984",SPHEROID["W
GS_1984",6378137.0,298.257223563]],PRIMEM["Greenwich",0.0],UNIT["Degree",0.017453292519943
3]],PROJECTION["Transverse_Mercator"],PARAMETER["false_easting",500000.0],PARAMETER["false
_northing",0.0],PARAMETER["central_meridian",30.0],PARAMETER["scale_factor",1.0],PARAMETER
["latitude_of_origin",0.0],UNIT["Meters",1.0]]

crs.to_epsg()      -> None
raster.describe(crs) -> WGS_1984_Transverse_Mercator (no EPSG equivalent)
UTM 36N (EPSG:32636) ölçek faktörü : 0.9996
bu projeksiyonun ölçek faktörü : 1.0
bu projeksiyonun merkez meridyeni : 30.0
```

Kanıt 14. WKT'nin tamamı ve `to_epsg()` sonucu. Projeksiyon merkez meridyen 30°E ile `kanit/B2-crs-ve-ortalama.txt` Transverse Mercator, ancak ölçek faktörü 1,0; UTM 36N 0,9996 kullanır.

Model UTM 36N **değildir** ve `to_epsg()` haklı olarak boş döner. Doğru kaynak, projenin kendi WKT'sidir. Aynı kanıt bloğu, hücre bazlı 0,117 m değerinin nasıl elde edildiğini de gösteriyor: ıslak hücrelerin ortalaması 0,1534 m, kuru hücreler sıfır sayılarak tüm gerçek hücrelere yayıldığında 0,1168 m.

5.7 Denenip ölçülerek kaldırılan bir yöntem

Taşkın kenarının tırtıklı görünümünü yumuşatmak için ıslak hücre merkezleri arasında doğrusal enterpolasyonla eğimli bir su yüzeyi denendi. Sonuç fiziksel olarak imkânsızdı: maksimum derinlik 20,4 m, 99. yüzdelik 11,4 m. Sebep, ıslak hücrelerin kopuk kümeler hâlinde olması ve üçgenlemenin vadinin bir yamacından diğerine uzanmasıydı. Seçenek “opsiyonel” diye teslim konmadı; kaldırıldı ve gerekçesi README'ye yazıldı.

BÖLÜM 6

Çıktının doğrulanması

“p05’i seçtim” bir doğrulama değildir. Doğrulama dört ayrı koldan yapıldı: kimlik zinciri, künye, bağımsız araç, ve girdiyle örtüşme.

6.1 Kimlik zinciri

Uygulama çıktı yazmadan önce sekiz kontrol çalıştırır ve sonuçları çalışma kaydına basar. Bir kontrol düşerse çıktı yazılmaz (çıkış kodu 8).

```
$ python -m pytest -q      (son tam kosu)
..... [ 90%]
..... [100%]
80 passed in 1.36s

$ python main.py --project <CASE_DATA> --use-existing-results --integrity full
(uretim ↵
    kosusu, tam kayıt)
q50depth 1.0.0 on Darwin 24.6.0, python 3.12.13
started 2026-08-29T23:56:55+03:00
[1/6] project <masaustu>/CASE_DATA/AKA_AFY_BAY_INPINAR_1/1_Modeller/
A_A_B_INPINAR.prj

Plans listed by the project file:
p03 A_A_B_INPINAR_UNSTEADY_2D      short id: INPINAR_Q500_UNSTEADY      geom g03
flow u ↵
01
p02 A_A_B_INPINAR_STEADY          short id: A_A_B_INPINAR              geom g01
flow f ↵
01
p01 A_A_B_INPINAR_STEADY          short id: A_A_B_INPINAR_STEADY      geom g01
flow f ↵
01
p04 A_A_B_INPINAR_Q500            short id: A_A_B_INPINAR_Q500        geom g03
flow u ↵
04
>> p05 A_A_B_INPINAR_Q50          short id: Q50                        geom g03
flow u ↵
05 <- matched on Plan Title, Short Identifier
p06 A_A_B_INPINAR_Q100            short id: Q100                      geom g03
flow u ↵
06
p07 A_A_B_INPINAR_Q1000           short id: Q1000                     geom g03
flow u ↵
08

[2/6] scenario Q50 -> p05 (pattern (?![0-9])[Qq]0*50(?![0-9]))
the project's own 'Current Plan=p05' agrees with the selection (not used to
decide)
[3/6] workspace skipped (--use-existing-results reads the project read-only)
note: p05 reads .\CBS\akarcay_debiler\akarcay_debi.dss (inflow), which does
not res ↵
olve in the project as delivered
note: p05 reads .\Q50\Q50.dss (output), which does not resolve in the project
as del ↵
ivered
note: p03: flow file u01 is not declared by the project
note: p04: boundary condition reads ..\..\akarcay_debi.dss, which does not
resolve
note: p05: boundary condition reads .\CBS\akarcay_debiler\akarcay_debi.dss,
```

```

which d ↵
  oes not resolve
    note: p06: boundary condition reads .\_CBS\akarcay_debiler\akarcay_debi.dss,
which d ↵
  oes not resolve
    note: p07: boundary condition reads .\_CBS\akarcay_debiler\akarcay_debi.dss,
which d ↵
  oes not resolve
[4/6] hec-ras      not run; using the results already in the project
      the notes above only matter for a real HEC-RAS run
[5/6] results      A_A_B_INPINAR.p05.hdf
      plan 'A_A_B_INPINAR_Q50', short id 'Q50', HEC-RAS 6.6 September 2024
      window 02May2025 01:00:00 to 02May2025 02:10:00
      2D areas: inpinar (5667 cells)
      terrain merge.Clone.vrt + 69 elevation modifications
      grid 4788 x 2756 @ 0.1 m
      wet cells 4165/5470, wet pixels 413858
      depth max 1.625 m, mean 0.130 m

Scenario verification:
[ok] results belong to the selected plan: results say plan file
'A_A_B_INPINAR.p05', sel ↵
    ected 'A_A_B_INPINAR.p05'
[ok] plan title agrees: results 'A_A_B_INPINAR_Q50' vs plan file
'A_A_B_INPINAR_Q50'
[ok] short identifier agrees: results 'Q50' vs plan file 'Q50'
[ok] results still carry the Q50 label: title='A_A_B_INPINAR_Q50', short_id='Q50'
[ok] geometry agrees: results 'A_A_B_INPINAR.g03' vs plan 'g03'
[ok] flow file agrees: results 'A_A_B_INPINAR.u05' vs plan 'u05'
[ok] the grid holds water: 413858 wet pixels, maximum depth 1.625 m
[ok] depth stays inside the modelled water surface: max water surface 1376.475 m,
terrai ↵
    n minimum 1322.219 m

[6/6] output      OUTPUT/q50_depth.tif (0.3 MB, WGS_1984_Transverse_Mercator (no
EPSG equ ↵
    ivalent))
      integrity    source unchanged (176 files, full check)

done in 1.0 s. Log: OUTPUT/run.log

```

Kanıt 15. Üretim koşusunun tamamı: plan listesi, seçim, veri notları, sonuç özeti, sekiz doğrulama kontrolü kanıt/G1-son-kosu.txt ve bütünlük raporu.

6.2 GeoTIFF künyesi ve bağımsız araç

Künye GeoTIFF'in içine gömülür, böylece dosya adına güvenmek gerekmez. Aşağıdaki kontrol **uygulamanın kendi kütüphanesiyle değil**, GDAL'in kendi komut satırı aracı gdalinfo ile yapılmıştır — yani çıktıyı üreten kodun dışından bakılmıştır.

```

$ gdalinfo OUTPUT/q50_depth.tif | sed -n '/Metadata/,/Image Structure/p'
Metadata:
  AREA_OR_POINT=Area
  COMPUTED_AT=2026-08-29T20:06:02+00:00
  FLOW_FILE=A_A_B_INPINAR.u05
  GEOMETRY=A_A_B_INPINAR.g03
  HEC_RAS_EXECUTED=False
  HEC_RAS_VERSION=HEC-RAS 6.6 September 2024
  PLAN_NUMBER=p05
  PLAN_SHORT_ID=Q50
  PLAN_TITLE=A_A_B_INPINAR_Q50
  QUANTITY=maximum water depth
  SCENARIO=Q50

```

```

SIMULATION_WINDOW=02May2025 01:00:00 to 02May2025 02:10:00
SOURCE_RESULTS=A_A_B_INPINAR.p05.hdf
SURFACE_METHOD=horizontal (cell-wise)
TERRAIN=merge.Clone.vrt
TERRAIN_MODIFICATIONS=69
TIFFTAG_SOFTWARE=q50depth 1.0.0
UNITS=m
Image Structure Metadata:

$ gdalinfo -stats ... (band istatistigi)
Band 1 Block=512x512 Type=Float32, ColorInterp=Gray
Minimum=0.000, Maximum=1.625, Mean=0.130, StdDev=0.120
NoData Value=-9999
  STATISTICS_MAXIMUM=1.6253662109375
  STATISTICS_MEAN=0.12953769747678
  STATISTICS_MINIMUM=0.0001220703125
  STATISTICS_STDDEV=0.11988914027003
  STATISTICS_VALID_PERCENT=3.136

```

Kanıt 16. gdalinfo ile bağımsız doğrulama: künye alanları ve bandın istatistikleri. kanit/F1-gdalinfo.txt
 STATISTICS_VALID_PERCENT=3.136, 413.858 / (4788 × 2756) oranına eşittir.

GDAL'in kendi hesapladığı maksimum (1,6254 m) ve ortalama (0,1295 m) değerleri, uygulamanın raporladığı 1,625 m ve 0,130 m ile örtüşüyor.

6.3 Girdi hidrografıyla örtüşme

Bir haritanın doğru olaya ait olduğunun en güçlü göstergesi, girdisiyle tutarlılığıdır. Planın okuduğu DSS serisi doğrudan okundu.

```

$ u05 hangi DSS serisini okuyor?
DSS Path = /AKA_AFY_BAY_INPINAR_1/AKC_126_YM/DEBI/02May2025/5Minute/Q50/
metin disa aktarimi bulundu: akarcay_debi.txt

$ Seri
ordinat sayisi : 15
tepe debi      : 1.69 M3/S
ilk deger      : 0.0 son deger: 0.02
baslangic      : None
aralik (dk)    : 5
tum degerler   : [0.0, 0.9, 1.69, 1.13, 0.56, 0.34, 0.19, 0.13, 0.12, 0.1, 0.08,
0.07, 0.05, 0.03, 0.02]

```

Kanıt 17. u05 akış dosyasının okuduğu DSS serisi ve ordinatların tamamı.

kanit/C3-hidrograf.txt

Üç bağımsız teyit birden çıkıyor:

1. **Süre.** 15 ordinat × 5 dakika = 70 dakika. Sonuç dosyasının bildirdiği simülasyon penceresi 02May2025 01:00:00 to 02May2025 02:10:00, yani tam 70 dakika.
2. **Etiket.** DSS yolunun F parçası (.../5Minute/Q50/) **Q50** yazıyor. Plan seçimi bu bilgiye bakmadan yapıldı, ama sonuç onu doğruluyor.
3. **Büyüklik.** Tepe debi 1,69 m³/s. Bu ölçekte bir olay için 1,6 m maksimum derinlik makul; 3,4 m ortalama derinlik (bkz. Bölüm 5.5) değildir.

6.4 Senaryoya özel davranmama

Boru hattının Q50'ye özel bir davranış barındırmadığı, aynı kodun başka tekerrürlerle çalıştırılmasıyla sınılandı.

```
$ Aynı boru hattı, senaryo etiketi degistirilerek uc kez:
```

senaryo	plan	islak hucre	islak piksel	maks (m)	ort (m)
Q50	p05	4165/5470	413858	1.625	0.130
Q100	p06	4232/5470	465321	1.643	0.135
Q1000	p07	4395/5470	690430	1.488	0.168

(tekerrur buyudukce islak alan artiyor; boru hattı tek senaryoya özel degil)

Kanıt 18. Aynı komut, yalnızca `--scenario` değiştirilerek üç kez.

kanit/F4-cok-senaryo.txt

Islak alan tekerrür büyüdükçe tek yönlü artıyor (413.858 → 465.321 → 690.430 piksel) ve her senaryo kendi planına gidiyor (p05, p06, p07).

6.5 Referans çıktı ile karşılaştırma

Toplantıda gösterilen görüntüde kanal boyunca kesintisiz, pürüzsüz bir şerit var; üretilen haritada ise parçalı taşkın lekeleri. İlk okuma “bizim harita eksik” yönündeydi.

Bu soru gözle değil, ölçülerek kapatıldı: projenin 1D geometrisi (A_A_B_INPINAR.g01) doğrudan geometri dosyasından okunup rasterin kendi coğrafi ölçeğine bindirildi. İş yapan araç `tools/reference_overlay.py` teslim paketindedir ve figürü yeniden üretir.

```
$ python tools/reference_overlay.py --geometry <CASE_DATA>/.../A_A_B_INPINAR.g01 \
--reference docs/referans_ekran_goruntusu.png -o docs/
referans_karsilastirma.png
A_A_B_INPINAR.g01
  centrelines      1 (47 points)
  cross sections   24 declared, 12 with a GIS cut line
  cut line points  [13, 17, 17, 16, 14, 13, 12, 23, 23, 13, 13, 16]
  centreline       x 577887.5..578339.8   y 4304334.6..4304452.0
  cut lines        x 577877.3..578349.8   y 4304318.8..4304468.4
q50_depth.tif
  raster extent    x 577883.1..578361.9   y 4304226.4..4304502.0
  cut lines inside the raster extent: 12/12
  figure           docs/referans_karsilastirma.png
```

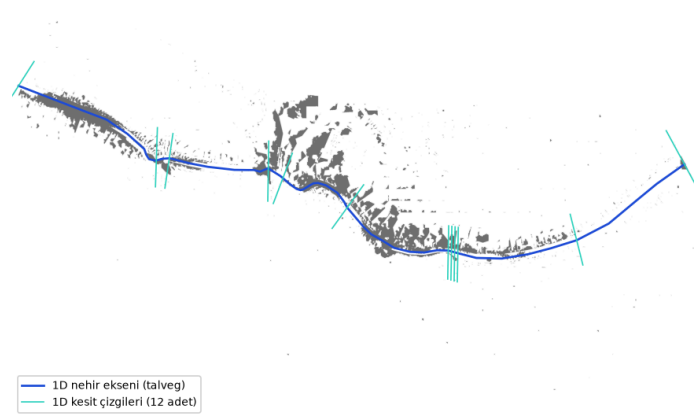
Kanıt 19. Bindirme aracının çıktısı. Nehir eksenini 47 noktadan oluşturuyor; geometri 24 kesit bildiriyor, bunların 12'sinin GIS kesit çizgisi var ve 12'si de rasterin sınırları içinde kalıyor.

kanit/F2-referans-bindirme.txt

Referans çıktı — toplantıda gösterilen görüntü



Üretilen Q50 maksimum derinlik haritası — aynı alan, aynı ölçek



Şekil 1. Üstte toplantıda gösterilen referans görüntü, altta üretilen Q50 maksimum derinlik haritası. Alt panele projenin 1D geometrisi bindirilmiştir: nehir ekseni (mavi) ve 12 kesit çizgisi (turkuaz). Üst paneldeki görüşme penceresi, üçüncü bir kişinin görüntüsünü içerdiği için maskelenmiştir; harita alanına dokunulmamıştır.

Nehir ekseni referanstaki şeridi birebir izliyor: aynı menderesler, aynı başlangıç ve bitiş noktası; sağ uçtaki huni biçimi de son kesitlerin genişlemesinden geliyor. **Yani o pürüzsüz şerit su değil, RASMapper görünümünde derinlik katmanının altında duran 1D nehir geometrisidir.** Suyu temsil eden koyu lekeler ise üretilen haritadaki ıslak alanlarla aynı konumdadır.

Karşılaştırmanın sınırı açıktır: toplantı görüntüsünün coğrafi referanslı bir kopyası elde bulunmadığından karşılaştırma piksel bazında değil, biçim ve konum düzeyindedir.

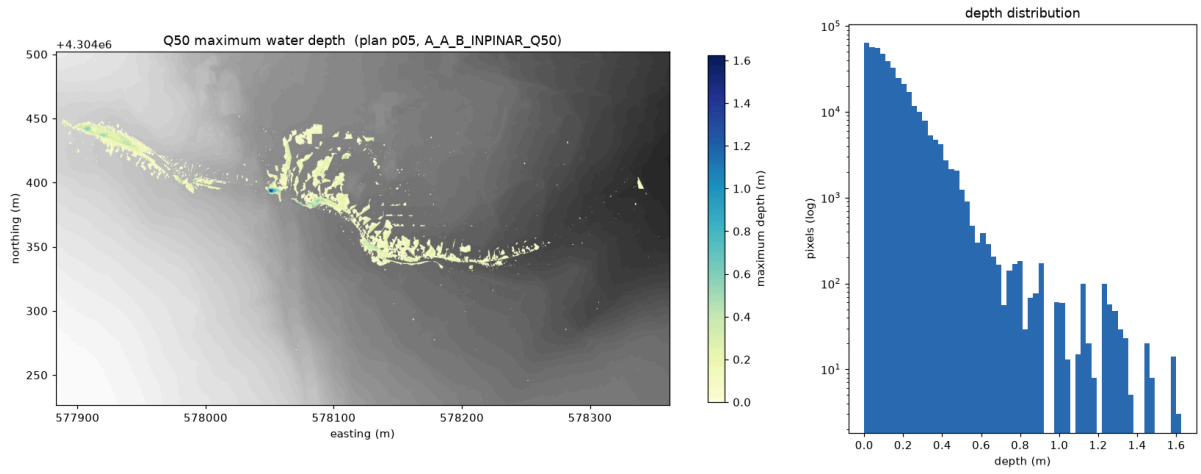
6.6 Çıktıya bakıldı

Üretmek ile doğrulamak aynı şey değildir. Çıktı tools/preview.py ile çizildi ve dağılımına bakıldı.

```
$ python tools/preview.py OUTPUT/q50_depth.tif --terrain <CASE_DATA>/.../
merge.Clone.vrt - ↵
  o docs/q50_depth_preview.png
OUTPUT/q50_depth.tif
  grid      4788 x 2756 @ 0.1 m
  extent    577883.1, 4304226.4 .. 578361.9, 4304502.0
  wet pixels 413858 (3.1% of the grid)
  wet area  4,139 m2
  depth     max 1.625 m, mean 0.130 m
  percentiles [0.098, 0.279, 0.516] m (50/90/99)
  scenario  Q50
  plan_number p05
  plan_title A_A_B_INPINAR_Q50
  plan_short_id Q50
  simulation_window 02May2025 01:00:00 to 02May2025 02:10:00
  preview    docs/q50_depth_preview.png
```

Kanıt 20. Önizleme aracının ölçtükleri: ızgara, kapsam, ıslak alan, yüzdelikler.

kanit/F3-onizleme.txt



Şekil 2. Q50 maksimum su derinliği (plan p05). Sol: arazi üzerine bindirilmiş derinlik haritası. Sağ: derinlik dağılımı (logaritmik ölçek). Islak alan 4.139 m^2 , medyan derinlik 0,098 m, 99. yüzdelik 0,516 m. Su, arazideki dere koridorunu takip ediyor.

BÖLÜM 7

Test süreci

7.1 Ne kadar test var, neyi kapsıyor

```
$ .venv/bin/python -m coverage run --source=q50depth -m pytest -q
..... [ 90%]
..... [100%]
80 passed in 1.85s

$ .venv/bin/python -m coverage report -m --sort=cover
Name                               Stmts  Miss  Cover
-----
q50depth/compute.py                 115    51    56%
q50depth/cli.py                     215    88    59%
q50depth/workspace.py                67    27    60%
q50depth/geometry.py                 93    21    77%
q50depth/hydrograph.py               88    14    84%
q50depth/results.py                  97    15    85%
q50depth/terrain.py                  62     6    90%
q50depth/project.py                 186    13    93%
q50depth/raster.py                   31     2    94%
q50depth/verify.py                   20     1    95%
q50depth/references.py                62     2    97%
q50depth/depth.py                   106     3    97%
q50depth/__init__.py                  1     0   100%
q50depth/errors.py                   21     0   100%
q50depth/logging_setup.py            23     0   100%
TOTAL                               1187   243    80%
```

Kanıt 21. Testlerin çalıştırılması ve modül bazında satır kapsamı.

kanit/E1-kapsam.txt

80 test, q50depth paketinin 1187 satırının %80'ini çalıştırıyor. Kapsamın en düşük olduğu iki modül anlam-
lıdır ve gizlenmemelidir:

Modül	Kapsam	Neden
compute.py	%56	HEC-RAS'ı çalıştıran modül; bu makinede HEC-RAS yok, çalıştırma yolu ancak Windows'ta yürütülebiliyor
cli.py	%59	Uçtan uca akışın büyük kısmı gerçek veriyle sınıyor; HEC-RAS gerektiren dallar açıkta kalıyor

Geri kalan modüllerin tamamı %77 ve üzerinde; derinlik hesabı (depth.py) %97, plan seçimi (project.py) %93.

7.2 Testlerin tam listesi

Test adları neyi iddia ettiklerini söyleyecek biçimde yazılmıştır; liste bu yüzden aynı zamanda bir davranış belgesidir.

```
$ .venv/bin/python -m pytest --collect-only -q (test envanteri)
test_cli.py::test_ras_dir_is_required_for_a_real_run
test_cli.py::test_missing_project_exits_cleanly
test_cli.py::test_unknown_scenario_exits_cleanly
test_cli.py::test_missing_results_exits_cleanly
test_cli.py::test_nothing_crashes_without_a_traceback
test_depth.py::test_ghost_cells_are_excluded
test_depth.py::test_dry_cell_on_a_raised_footprint_is_not_wet
test_depth.py::test_depth_is_water_surface_minus_terrain
test_depth.py::test_building_modification_is_applied_to_the_terrain
```

```

test_depth.py::test_without_the_dry_cell_rule_the_building_would_flood
test_depth.py::test_building_inside_a_wet_cell_stays_dry
test_depth.py::test_min_depth_filters_thin_films
test_depth.py::test_coarser_resolution_still_produces_water
test_depth.py::test_unsupported_modification_type_is_refused
test_depth.py::test_missing_terrain_raster_is_refused
test_hydrograph.py::test_series_is_matched_across_different_spellings
test_hydrograph.py::test_return_periods_are_not_confused
test_hydrograph.py::test_hours_follow_the_declared_interval
test_hydrograph.py::test_missing_series_is_refused
test_hydrograph.py::test_ambiguous_series_is_refused
test_hydrograph.py::test_a_short_block_is_refused
test_hydrograph.py::test_unknown_interval_is_refused
test_hydrograph.py::test_dss_pathname_is_read_from_the_flow_file
test_hydrograph.py::test_text_export_is_found_anywhere_in_the_project
test_prepare.py::test_defects_name_the_plan_and_the_reason
test_prepare.py::test_reduced_project_declares_only_the_selected_plan
test_prepare.py::test_geometry_and_flow_declarations_survive_the_reduction
test_prepare.py::test_reduced_project_keeps_crlf
test_prepare.py::test_reduced_project_drops_dss_entries_that_cannot_be_opened
test_prepare.py::test_reduced_project_adds_a_declaration_the_project_forgot
test_prepare.py::test_stale_results_are_removed
test_prepare.py::test_verify_results_rejects_a_stub_file_and_quotes_the_hecras_log
test_prepare.py::test_verify_results_reports_a_run_that_wrote_nothing
test_prepare.py::test_verify_results_accepts_a_finished_run
test_prepare.py::test_plan_flag_is_changed_in_place
test_prepare.py::test_plan_flag_reports_a_key_that_is_not_there
test_prepare.py::test_compute_messages_file_is_preferred_over_the_run_banner
test_prepare.py::test_compute_messages_are_read_from_the_results_file_when_no_log_was_writ
↵
ten
test_prepare.py::test_missing_preprocessed_tables_are_detected
test_prepare.py::test_geometry_is_rebuilt_from_the_results_file
test_prepare.py::test_rebuild_refuses_when_the_results_file_is_no_better
test_prepare.py::test_hecras_geometry_tool_is_reported_missing_not_guessed
test_prepare.py::test_terrain_timestamp_is_set_from_the_geometry
test_prepare.py::test_terrain_alignment_is_skipped_when_nothing_is_recorded
test_project.py::test_finds_the_hecras_prj_and_ignores_coordinate_files
test_project.py::test_project_file_is_found_from_a_grandparent_folder
test_project.py::test_missing_project_is_reported
test_project.py::test_plan_list_comes_from_the_project_file_not_the_folder
test_project.py::test_short_identifier_padding_is_stripped
test_project.py::test_q50_selects_exactly_p05
test_project.py::test_other_return_periods_resolve_to_their_own_plan[Q50-p05]
test_project.py::test_other_return_periods_resolve_to_their_own_plan[Q100-p06]
test_project.py::test_other_return_periods_resolve_to_their_own_plan[Q1000-p07]
test_project.py::test_q500_is_genuinely_ambiguous_in_this_project
test_project.py::test_unknown_scenario_fails_loudly
test_project.py::test_ambiguous_scenario_fails_loudly
test_project.py::TestScenarioPattern::test_matches[Q50]
test_project.py::TestScenarioPattern::test_matches[A_A_B_INPINAR_Q50]
test_project.py::TestScenarioPattern::test_matches[Q050]
test_project.py::TestScenarioPattern::test_matches[x Q50 y]
test_project.py::TestScenarioPattern::test_matches[q50]
test_project.py::TestScenarioPattern::test_does_not_match[Q500]
test_project.py::TestScenarioPattern::test_does_not_match[INPINAR_Q500_UNSTEADY]
test_project.py::TestScenarioPattern::test_does_not_match[Q5000]
test_project.py::TestScenarioPattern::test_does_not_match[Q100]
test_project.py::TestScenarioPattern::test_does_not_match[Q1000]
test_project.py::TestScenarioPattern::test_does_not_match[Q5]
test_project.py::TestScenarioPattern::test_naive_substring_search_would_be_wrong
test_project.py::TestScenarioPattern::test_rejects_nonsense
test_real_data.py::test_selects_p05_and_nothing_else
test_real_data.py::test_the_stray_backup_plan_is_on_disk_but_not_selected

```



```

test_real_data.py::test_full_pipeline_on_existing_results
test_real_data.py::test_cli_writes_a_readable_geotiff
test_references.py::test_collect_separates_inflow_from_output
test_references.py::test_repair_puts_the_inflow_file_where_the_model_looks
test_references.py::test_repair_creates_the_output_folder_but_not_the_file
test_references.py::test_everything_resolves_after_repair
test_references.py::test_repair_is_a_no_op_on_a_second_run
test_references.py::test_missing_inflow_file_is_refused
test_references.py::test_ambiguous_inflow_file_is_refused

$ dosya basına test sayisi
25 test_project.py
20 test_prepare.py
10 test_depth.py
9 test_hydrograph.py
7 test_references.py
5 test_cli.py
4 test_real_data.py

```

Kanıt 22. Toplanan 80 testin tamamı ve dosya başına dağılımı.

kanit/E2-test-envanteri.txt

7.3 Hata yolları ve çıkış kodları

Belgenin 3. maddesi, eksik veya beklenmeyen bir proje durumunun uygulamayı kontrolsüz kapatmamasını istiyor. Beklenebilir her hata tek satırlık bir mesaj, çoğu zaman bir ipucu ve hata sınıfına özgü bir çıkış kodu üretir. Yığın izi yalnızca gerçek bir yazılım hatasında görünür.

```

### bos klasor
$ python main.py --project <gecici>/bos --use-existing-results --output <gecici>/
e.tif
q50depth 1.0.0 on Darwin 24.6.0, python 3.12.13
started 2026-08-29T23:50:27+03:00
ERROR: No HEC-RAS project file found under <gecici>/bos
      A HEC-RAS .prj file contains 'Plan File=' lines. Coordinate system .prj files
do not
      t; those are ignored on purpose.
-> cikis kodu: 3

### olmayan senaryo
$ python main.py --project <masaustu>/CASE_DATA --scenario Q77 --use-existing-
results --ou
      tput <gecici>/e.tif
p04 title='A_A_B_INPINAR_Q500' short_id='A_A_B_INPINAR_Q500'
p05 title='A_A_B_INPINAR_Q50' short_id='Q50'
p06 title='A_A_B_INPINAR_Q100' short_id='Q100'
p07 title='A_A_B_INPINAR_Q1000' short_id='Q1000'
-> cikis kodu: 4

### coklu eslesme
$ python main.py --project <masaustu>/CASE_DATA --scenario Q500 --use-existing-
results --o
      utput <gecici>/e.tif
ERROR: Scenario Q500 matches 2 plans, which is ambiguous:
p03 title='A_A_B_INPINAR_UNSTEADY_2D' short_id='INPINAR_Q500_UNSTEADY'
p04 title='A_A_B_INPINAR_Q500' short_id='A_A_B_INPINAR_Q500'
      Refine --scenario, or fix the plan titles in the project.
-> cikis kodu: 4

### gecersiz senaryo adi
$ python main.py --project <masaustu>/CASE_DATA --scenario ABC --use-existing-
results --ou
      tput <gecici>/e.tif
q50depth 1.0.0 on Darwin 24.6.0, python 3.12.13

```

```

started 2026-08-29T23:50:27+03:00
[1/6] project <masaustu>/CASE_DATA/AKA_AFY_BAY_INPINAR_1/1_Modeller/
A_A_B_INPINAR.prj
ERROR: Scenario must look like 'Q50', got 'ABC'.
-> cikis kodu: 4

### eksik --ras-dir
$ python main.py --project <masaustu>/CASE_DATA --output <gecici>/e.tif
q50depth 1.0.0 on Darwin 24.6.0, python 3.12.13
started 2026-08-29T23:50:27+03:00
ERROR: --ras-dir is required to run HEC-RAS.
    Give the HEC-RAS installation folder, or pass --use-existing-results to
    rebuild the ↵
    raster from results that are already in the project.
-> cikis kodu: 2

### hesaplanmamis plan
$ python main.py --project <masaustu>/CASE_DATA --scenario Q500 --use-existing-
results --o ↵
    utput <gecici>/e.tif
ERROR: Scenario Q500 matches 2 plans, which is ambiguous:
    p03 title='A_A_B_INPINAR_UNSTEADY_2D' short_id='INPINAR_Q500_UNSTEADY'
    p04 title='A_A_B_INPINAR_Q500' short_id='A_A_B_INPINAR_Q500'
    Refine --scenario, or fix the plan titles in the project.
-> cikis kodu: 4

```

Kanıt 23. Beş hata durumu gerçekten çalıştırıldı; her birinin mesajı ve çıkış kodu.

kanit/E3-hata-yollari.txt

Sonuç dosyası bulunmayan bir proje için ayrı bir kontrollü deney yapıldı: proje metin dosyaları bir kopyaya alındı, .hdf dosyaları bilinçli olarak dışarıda bırakıldı.

```

$ Kontrollu deney: sonuc dosyasi (p05.hdf) OLMAYAN bir proje kopyasi
$ ls kopya/.../1_Modeller/ | head
A_A_B_INPINAR.f01
A_A_B_INPINAR.g01
A_A_B_INPINAR.g02
A_A_B_INPINAR.g03
A_A_B_INPINAR.p01
A_A_B_INPINAR.p02
    (.hdf dosyasi bilincli olarak kopyalanmadi)

$ python main.py --project <kopya> --use-existing-results
    the notes above only matter for a real HEC-RAS run

ERROR: No results file at <gecici>/kopya/AKA_AFY_BAY_INPINAR_1/1_Modeller/
A_A_B_INPINAR.p0 ↵
    5.hdf
    The plan has not been computed yet. Run without --use-existing-results to let
    HEC-R ↵
    AS produce it.
-> cikis kodu:

```

Kanıt 24. Hesaplanmamış plan durumu; çıkış kodu 6.

kanit/E4-eksik-sonuc.txt

Çıkış kodu haritası q50depth/errors.py içinde istisna sınıflarına bağlıdır:

Kod	Durum	Kod	Durum
0	Başarılı	6	Sonuç dosyası eksik ya da yarım
2	Argüman hatası	7	Arazi çözülemedi
3	Proje bulunamadı	8	Senaryo doğrulaması düştü
4	Senaryo sıfır/çoklu eşleşti	70	Beklenmeyen hata (kayda tam iz)
5	HEC-RAS başlatılamadı / hesap başarısız	130	Kullanıcı iptali

7.4 Tekrarlanabilirlik

Belgenin 4. maddesi, teslim edilen talimatlar izlenerek çıktının yeniden üretilebilmesini istiyor. Bu, “çalışıyor” demekten daha kuvvetli biçimde sınıandı: aynı komut iki kez çalıştırıldı ve piksel verisinin SHA-256 özeti karşılaştırıldı.

```
$ Aynı komut iki kez, iki ayrı dosyaya:
  kosu 1 bitti
  kosu 2 bitti

$ Piksel verisinin SHA-256'si (kunya disı, ham dizi):
  kosu 1: b0620589e1e6e400b26cfa51415339102667520d39fd7ddfff83f0abb73e3567
shape=(2756, 4788) dtype=float32
  kosu 2: b0620589e1e6e400b26cfa51415339102667520d39fd7ddfff83f0abb73e3567
shape=(2756, 4788) dtype=float32

  diziler birebir esit : True
  transform esit      : True
  crs esit            : True

  TESLİM EDİLEN dosya ile karsilastirma:
  piksel verisi esit   : True
  kunyede farkli alan : ['COMPUTED_AT']
    COMPUTED_AT: yeni='2026-08-29T20:48:32+00:00'
  teslim='2026-08-29T20:06:02+00:00'
```

Kanıt 25. İki bağımsız koşunun piksel verisi ve teslim edilen dosyayla karşılaştırması.

kanit/D1-tekrarlanabilirlik.txt

İki koşunun özeti aynı; teslim edilen OUTPUT/q50_depth.tif ile de aynı. Künyede değişen tek alan COMPUTED_AT — koşunun zaman damgası. Yani çıktı bit düzeyinde belirlenimlidir.

7.5 Testlerin teslim edilen çıktıya dokunmaması

Bu denetimde bulunan tek gerçek kusur burada ortaya çıktı ve düzeltildi. Teslim kalemi olan OUTPUT/run.log, haritayı üreten koşunun değil, bir **pytest** koşusunun yazdığı altı satırlık hata kaydıydı: test_missing_project_exits_cleanly --output vermediği için varsayılan OUTPUT/run.log yoluna yazıyor ve her test koşusu teslim kalemini eziyordu. Test kendi geçici dizinine yönlendirildi, kayıt gerçek üretim koşusundan yeniden üretildi; Bölüm 6 bölümündeki kanıt bloğu o koşunun kaydıdır.

BÖLÜM 8

HEC-RAS motorunun çalışmaması

Kanıt sınıfı (C). Bu bölümdeki bulgular Windows 11 + HEC-RAS 6.6 üzerinde 29 Ağustos 2026'da ölçülmüştür. HEC-RAS bir Windows programı olduğu için bu ölçümler bu raporun hazırlandığı maki-nede tekrarlanamaz. HEC-RAS'ın ham hesap günlüğü dosyası teslim paketinde **yoktur**; aşağıdaki hata metni o oturumdan aktarılmıştır. Bu bölümdeki hiçbir ifade, ölçülmüş kanıt bloklarıyla aynı düzeyde sunulmamaktadır.

Belgenin 3. maddesi haritanın HEC-RAS proje sonuçları kullanılarak üretilmesini istiyor; uygulama bunu yapıyor. Planı yeniden hesaplama yolu da kodlanmıştır, ancak teslim edilen veri o kurulumda hesaplanamadı.

Bölüm 4 bölümündeki onarımlardan sonra HEC-RAS geometriyi sorunsuz işliyor, ardından unsteady motoru başlar başlamaz düşüyor:

```
Performing Unsteady Flow Simulation HEC-RAS 6.6 September 2024
forrtl: severe (157): Program Exception - access violation
Image          PC          Routine          Line    Source
RasUnsteady.exe 00007FF64F701486 READ_UN_HDF_STRUC 330
Read_UN_HDF_STRUC_GRP.for
RasUnsteady.exe 00007FF64F8C0CC3 SNETREAL2          179 Snetreal2.for
RasUnsteady.exe 00007FF64F9D561A UNET_START          144 Unet_start.for
Error with program: RasUnsteady.exe Exit Code = 157
```

Windows oturumundan aktarılan hata metni – ölçülmüş kanıt bloğu değildir.

Neden, tahminle değil, tek tek değişken sabitlenerek daraltıldı:

Deneme	Sonuç
Teslim edildiği gibi (yalnız DSS yolu onarılmış)	Çöküyor
+ proje tek plana indirgenmiş	Çöküyor
+ hidrograf akış dosyasına gömülmüş, DSS bağımlılığı kaldırılmış	Çöküyor
+ eksik geometri tabloları geri konmuş, koşu sonrası yerinde olduğu doğrulanmış	Çöküyor
+ RasProcess.exe CompleteGeometry çalıştırılmış	Çöküyor
+ RASMapper kapatılmış, arazi zaman damgası hizalanmış	Çöküyor
HEC-RAS arayüzünde elle açılıp çalıştırılmış	Çöküyor

Son satır belirleyicidir: çöküş uygulama kaynaklı değildir, aynı hata proje HEC-RAS arayüzünde elle açılıp çalıştırıldığında da oluşmaktadır. Geriye kalan açıklama, modelin 2D hidrolik bağlantılarında HEC-RAS'ın kendi hatası ya da teslim paketinde henüz tespit edilememiş bir eksiktir. Doğrulanabilir bir cevap üretilemediği için konu tahminle kapatılmamıştır.

Uygulamanın bu duruma verdiği tepki önemlidir: çalıştırıcı kütüphane hesabı “başarılı” olarak raporladı, uygulama buna **inanmadı**, sonuç dosyasını kendisi açtı, yarım olduğunu gördü ve HEC-RAS'ın kendi hesap günlüğünü alıntılarla durdu. Bu davranış üç testle sabitlenmiştir:

- test_verify_results_rejects_a_stub_file_and_quotes_the_hec_ras_log
- test_verify_results_reports_a_run_that_wrote_nothing
- test_verify_results_accepts_a_finished_run

Bu nedenle teslim edilen çıktı, projenin **kendi hesaplanmış sonuçlarından** üretilmiştir. Hazır bir raster kopyalanmamıştır; harita her çalıştırmada HEC-RAS'ın sonuç dosyasından yeniden hesaplanır. GeoTIFF künyesindeki HEC_RAS_EXECUTED=False etiketi bu durumu açıkça bildirir. Denenmemiş kalan iki seçenek vardır ve dürüstlük gereği yazılmaktadır: --cores 1 ve --runner controller.

BÖLÜM 9

Bilinen sınırlamalar ve kanıt boşlukları

Bu bölüm, raporun kendi zayıf noktalarını sayar.

Konu	Durum
HEC-RAS motorunun çalıştırılması	Teslim edilen veriyle çöküyor; kanıtı (C) sınıfı, ham hesap günlüğü pakette yok. Bu boşluğu kapatmanın yolu, Windows'ta bir koşu daha yapıp <code>Compute Messages</code> dosyasını pakete eklemektir.
Referans karşılaştırması	Toplantı görüntüsünün coğrafi referanslı kopyası olmadığı için piksel bazında değil, biçim ve konum düzeyinde. Bindirme aracı yeniden üretilebilir (A).
Su yüzeyi modeli	Hücre içinde sabit kabul edilir; modelin çözdüğü büyüklük budur. Sonuç, taşkın kenarında tırtıklı bir sınırdır.
Akış tipi	Yalnızca 2D akış alanları işlenir. 1D kesit sonuçlarından derinlik ızgarası üretilmez; böyle bir planla çalıştırılırsa anlaşılır hata verilir.
Arazi değişikliği tipi	Yalnızca <code>Add</code> destekleniyor. Veri setinde yalnızca bu tip var (Kanıt 12); başka bir tiple karşılaşılsa sessizce yanlış harita üretmek yerine durulur — <code>test_unsupported_modification_type_is_refused</code> .
Kapsam boşluğu	<code>compute.py</code> %56 kapsamda; HEC-RAS gerektiren dallar bu makinede yürütülemiyor.

BÖLÜM 10

Yapay zekâ kullanımı

Belgenin 7. maddesi, yapay zekâ destekli araç kullanımını serbest bırakmakta ve hangi araçların hangi amaçlarla kullanıldığının belirtilmesini istemektedir.

Kullanılan araç: **Claude (Anthropic) – Opus 5**, Claude Code komut satırı ajanı. Başka bir üretken yapay zekâ aracı kullanılmamıştır.

Aşama	Kullanım	Doğrulama biçimi
Veri keşfi	Sonuç dosyasındaki 439 HDF5 yolunun taranması, veri setlerinin bulunması	Kod çalıştırılarak; şekil, birim ve değer aralıkları basıldı (Kanıt 3, 4)
Analiz	Bina kot değişikliği ve kuru hücre etkilerinin bulunması	İki etki de çarpım tablosuyla ölçüldü (Kanıt 13)
Kütüphane	<code>ras-commander</code> API imzalarının çıkarılması	Paket kuruldu, <code>inspect.signature</code> ile okundu (Kanıt 8, 9)
Teşhis	Motor çöküşünün nedeninin daraltılması	Denetim aracı yazıldı; her hipotez ölçümle elendi
Kod ve test	Modüllerin ve 80 testin yazılması	Gerçek veri üzerinde uçtan uca çalıştırıldı (Kanıt 15, 21)
Bu rapor	Metnin yazılması ve dizgisi	Her sayı, rapor hazırlanırken komut çalıştırılarak yeniden ölçüldü; tutmayan üç değer düzeltildi

Uygulanan kurallar: doğrulanmamış hiçbir kütüphane çağırısı koda alınmadı; her teknik iddia veriye karşı sınılandı; çalışmayan bir özellik (eğimli su yüzeyi) ölçülüp kaldırıldı; doğrulanamayan adımlar açıkça işaretlendi. Kodun tamamı okunmuş, çalıştırılmış ve savunulabilir durumdadır.

BÖLÜM 11

Kanıt dizini

Bu rapordaki her kanıt bloğunun ham çıktısı teslim paketinde uygulama/docs/rapor/kanit/ klasöründedir. Aşağıdaki tablo hangi dosyanın hangi komutla üretildiğini gösterir; komutlar dosyaların ilk satırlarında da yazılıdır.

Dosya	Üreten komut
A1-ortam.txt	sw_vers; python -VV; import h5py,numpy,rasterio; git log -1
A2-veri-envanteri.txt	find CASE_DATA -type f wc -l; du -sh; uzantı sayımı
A3-hdf-yollari.txt	h5py: f.visit ile 439 yolun taranması
A4-maksimum-su-kotu.txt	h5py: Summary Output veri setleri ve öznitelikleri
B1-mesh-ve-kimlik.txt	q50depth.results.load(); real_cells / wet_cells sayımı
B2-crs-ve-ortalama.txt	q50depth.raster.crs_from_wkt(); to_epsg()
B3-arazi.txt	q50depth.terrain.resolve(); modification sayımı
B5-carpim-tablosu.txt	q50depth.depth.build() dört yapılandırma ile
C1-plan-kaynagi.txt	grep 'Plan File=' A_A_B_INPINAR.prj; head Backup.p01
C2-regex.txt	q50depth.project.scenario_pattern('Q50')
C3-hidrograf.txt	q50depth.hydrograph.read_series()
D1-tekrarlanabilirlik.txt	main.py iki kez; hashlib.sha256(piksel verisi)
D2-butunluk.txt	q50depth.workspace.manifest/compare, kopya üzerinde
E1-kapsam.txt	coverage run --source=q50depth -m pytest; coverage report
E2-test-envanteri.txt	pytest --collect-only -q
E3-hata-yollari.txt	main.py beş hatalı girdiyle; \$? okunarak
E4-eksik-sonuc.txt	main.py, .hdf içermeyen proje kopyası üzerinde
F1-gdalinfo.txt	gdalinfo -stats OUTPUT/q50_depth.tif
F2-referans-bindirme.txt	tools/reference_overlay.py --geometry ...g01
F3-onizleme.txt	tools/preview.py OUTPUT/q50_depth.tif --terrain ...
F4-cok-senaryo.txt	main.py --scenario Q50 Q100 Q1000
G1-son-kosu.txt	pytest -q; main.py --use-existing-results --integrity full
H1-ras-commander.txt	inspect.signature(rc.RasCmdr.compute_plan) vb.
H2-api-uyum.txt	imza parametreleri ile compute.py kullanımının karşılaştırması
veri-denetimi-ciktisi.txt	tools/audit_project.py --project CASE_DATA